Retrieval-Augmented Generation

Besher Alkurdi

What Does an LLM Know?

Everything it saw during training. Nothing else.

Its knowledge is **frozen** at the training cutoff date.

It cannot tell you what it doesn’t know.

The Hallucination Problem

> The capital of Syria was founded in 2003…

The model does not retrieve facts. It **generates** plausible text.

Three Fundamental Limits

Limit	Example
Knowledge cutoff	*Who won the 2025 election?*
No private data	*Summarize our Q3 report*
No sources	*Where did you get that?*

Scaling parameters does not solve any of these.

What If the Model Could Look Things Up?

Retrieve relevant documents **at inference time**

Condition generation on those documents

This is **Retrieval-Augmented Generation**

Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*

The Pipeline

```
Query → Retriever → Top-k Documents → LLM → Answer
```

The retriever determines what the model sees.

Sparse Retrieval: BM25

Match documents by **term frequency** and **inverse document frequency**

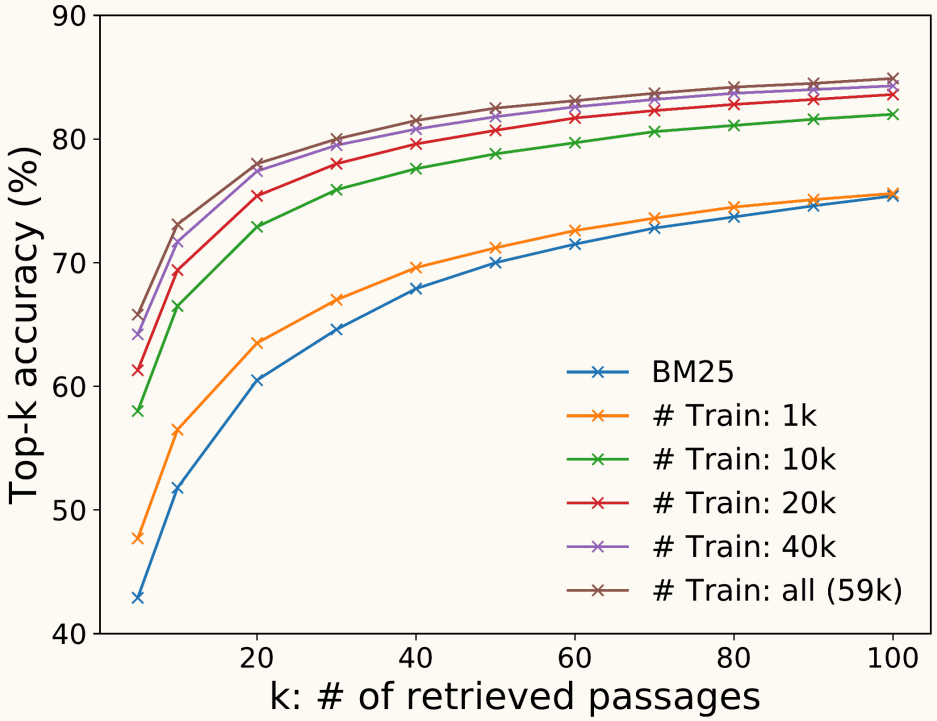
- Fast, no training needed
- Works well for keyword-heavy queries
- Used in over half of RAG systems as a baseline

Fails on: paraphrases, synonyms, semantic queries

Dense Retrieval

Encode queries and documents into **vector embeddings**

Compare by cosine similarity in a shared space



Karpukhin et al. (2020)

Dense Retrieval: Key Models

Model	Key Idea	Year
DPR	Dual BERT encoders	2020
ColBERT	Late interaction (token-level)	2020
Sentence-BERT	Siamese networks for embeddings	2019
ANCE	Hard negative mining during training	2020

The Vocabulary Gap

Query: *heart attack symptoms*

Relevant doc: *Clinical presentation of myocardial infarction…*

BM25 score: **0** (no shared terms)

Dense retriever: **high similarity** (same semantic space)

But dense retrievers can miss exact entity names.

Hybrid Retrieval

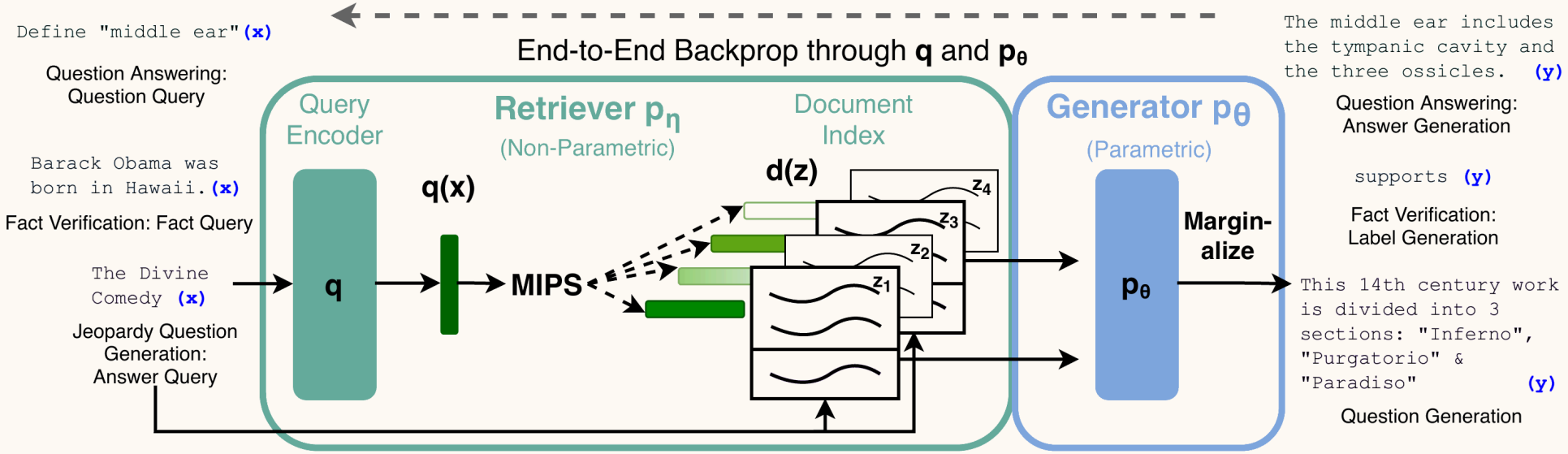
Combine sparse + dense retrieval scores

Reciprocal Rank Fusion (RRF): merge ranked lists from both

Consistently outperforms either method alone across benchmarks

Sawarkar et al. (2024), *Blended RAG*

The Original RAG Architecture



Lewis et al. (2020)

Two Flavors of RAG

RAG-Sequence: same documents for the entire output

RAG-Token: different documents for each generated token

Both marginalize over retrieved documents

Lewis et al. (2020)

Evolution of Architectures

Model	Key Innovation	Year
REALM	Retrieval during pre-training	2020
RAG	Seq2seq + dense retriever	2020
FiD	Encode passages separately, fuse in decoder	2021
RETRO	Chunked cross-attention, 2T token DB	2022
Atlas	11B params beats 540B with retrieval	2022

The Scaling Debate

RETRO (7.5B params) $\approx$ GPT-3 (175B params)

Atlas (11B params) > PaLM (540B params) on NQ

Retrieval is cheaper than memorization.

And you can update the knowledge base without retraining.

The Modern Paradigm

Early RAG: jointly train retriever + generator (expensive)

Modern RAG: **frozen LLM** + retriever pipeline

Just prepend retrieved documents to the prompt

No architectural changes to the LLM needed

Why Naive RAG Fails

Single-shot retrieve-then-generate has problems:

- Retrieves irrelevant documents for ambiguous queries
- Cannot handle multi-hop reasoning
- No quality control on what was retrieved

Query Transformation

Rewrite the query before retrieval

HyDE: generate a hypothetical answer, use *that* as the query

Multi-query: expand one query into multiple variations

Step-back prompting: generalize the query for broader context

Iterative Retrieval

FLARE: generate sentence by sentence, retrieve when uncertain

Low confidence token? Pause, retrieve, regenerate.

ITER-RETGEN: use the model’s own output to refine retrieval queries

Self-RAG

The model decides **when** to retrieve and **whether** to trust what it found

Special **reflection tokens** during generation:

- *Should I retrieve?*
- *Is this passage relevant?*
- *Is my answer supported?*

Asai et al. (2023)

Corrective RAG (CRAG)

Evaluate retrieval quality **before** generation

Lightweight evaluator scores retrieved docs: correct / incorrect / ambiguous

If insufficient: rewrite query or fall back to web search

Beyond Text

Documents contain tables, figures, diagrams, charts

Standard text embeddings cannot represent visual content

Two approaches: extract text from images, or embed images directly

Multimodal Embeddings

CLIP: joint text-image embedding space

Train two encoders (image + text) on 400M image-text pairs

Query with text, retrieve images. Or vice versa.

Multimodal RAG Pipeline

```
Query → Text Retriever  → [text chunks]  ⎤
                                         ⎥→ Multimodal LLM → Answer
Query → Image Retriever → [images/figs]  ⎦
```

Vision-language models (GPT-4o, Gemini) can reason over both

Vector Databases

Store and search embeddings at scale

Tool	Type	Key Feature
FAISS	Library (Meta)	Billion-scale ANN search
ChromaDB	Embedded DB	Simple Python API
Pinecone	Managed service	Serverless, hybrid search
Weaviate	Open-source DB	Native multimodal support

RAG Frameworks

Framework	Strength
LangChain	Largest ecosystem, most integrations
LlamaIndex	Best for structured data + indexing
Haystack	Production pipelines (by deepset)

All support: chunking, embedding, retrieval, reranking, generation

Embedding Models

Model	Dims	Context
`text-embedding-3-large` (OpenAI)	3072	8K tokens
`all-MiniLM-L6-v2` (SBERT)	384	256 tokens
`bge-large-en-v1.5` (BAAI)	1024	512 tokens
`nomic-embed-text`	768	8K tokens

Check the MTEB Leaderboard for current rankings

What Can Go Wrong

Retrieval failures: wrong documents, missing documents

Faithfulness failures: answer contradicts the retrieved context

Relevance failures: answer does not address the question

Hallucination: answer invents facts not in any retrieved document

RAGAS Framework

Three core metrics, no ground truth needed:

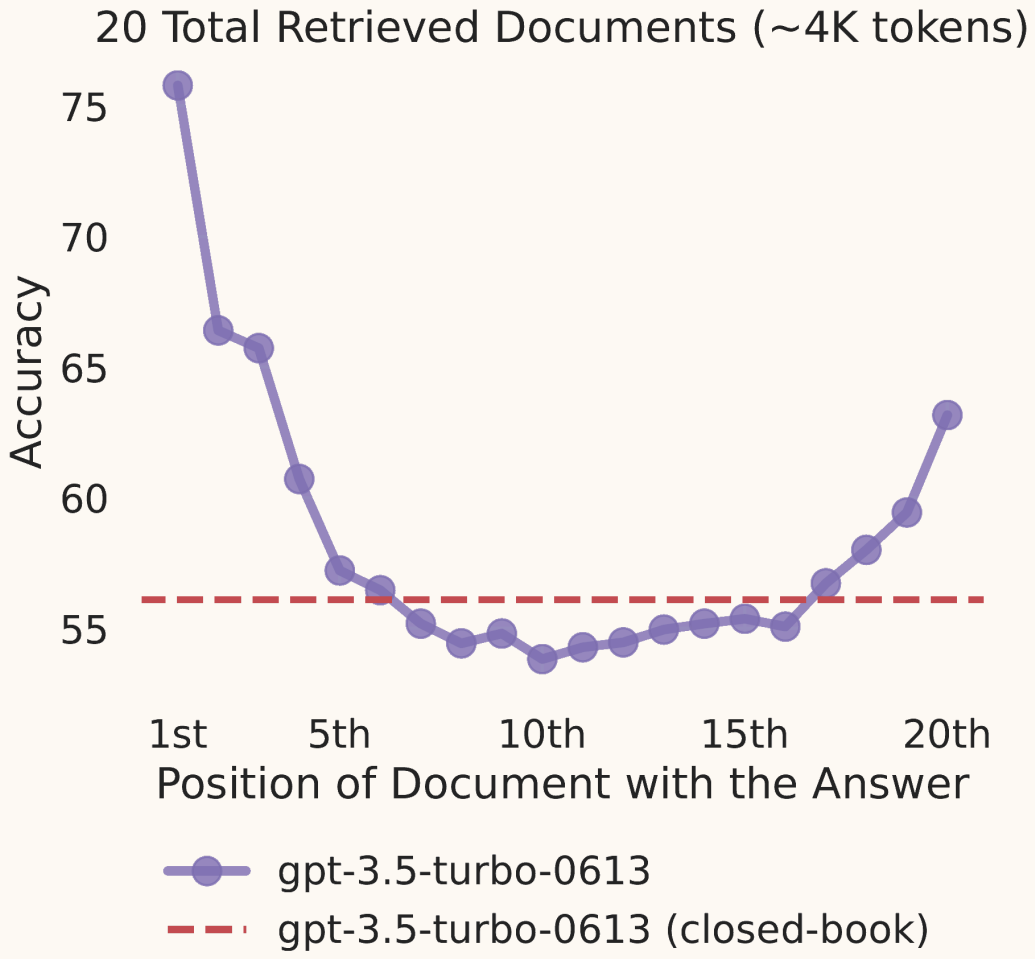
Context Relevance: are the retrieved docs relevant to the query?

Faithfulness: is the answer grounded in the retrieved context?

Answer Relevance: does the answer actually address the query?

Shahul et al. (2023)

Lost in the Middle



Models attend to the **beginning** and **end** of context.

RAG in Practice

```
1. Chunk documents (512-1024 tokens)
2. Embed chunks → vector store
3. At query time:
   a. Embed the query
   b. Retrieve top-k chunks
   c. (Optional) Rerank
   d. Concatenate query + chunks → LLM
4. Generate answer with citations
```

What We Covered

- LLMs have frozen, uncitable knowledge
- RAG = retrieval + generation
- Sparse, dense, and hybrid retrieval
- Architectures: RAG, FiD, RETRO, Atlas
- Advanced: query rewriting, iterative, Self-RAG, CRAG
- Multimodal embeddings extend RAG beyond text
- Evaluate with RAGAS: context relevance, faithfulness, answer relevance